

Handoff — Modal templates roadmap

Last updated: 2026-05-31. Sprint 0 + Sprint 1 + Sprint 2 shipped. Sprint 3 IN PROGRESS — templateId gene foundation + storefront render pipeline/preview harness landed (commits c6a69fb, ad01570). See Sprint 3 status for what's left.

Working agreement (read CLAUDE.md too)

- Solo dev, commit directly to `main`, push when done, user pulls + tests
- No worktrees, no feature branches, no PRs
- Terse responses (caveman style). No preamble. Show diffs not walkthroughs.
- User has zero customers — never invent uplift %, CVR, recovered \$, testimonials

What this work is about

User asked: review the AI mode against the spec in `Resparq AI 5.26.26.pdf`, then build a roadmap of features. Roadmap converged on a multi-sprint plan to:

1. Fix real bugs (Starter gate)
2. Polish UX (progress bar theming, discount badge)
3. Build the 8 modal-design templates the marketing deck already promised but never existed in code
4. Wire templates to a manual picker + live preview
5. Then a "catch" component for dismissal recovery
6. Then AI integration (templateId as gene, meta-learning, Pro lift upsell)

Architecture decisions made (don't relitigate)

- **Modal templates = 8, not 10.** Cut "Free Shipping Bar" (economic conflict with merchant's existing shipping threshold app — kills their AOV strategy) and "Minimal Text" (too close to Classic Card). Final set: Classic Card, Top Banner, Bottom Sheet, Coupon Ticket, Split Hero, Timer-Front, Testimonial, Scratch Reveal.
- **Per-archetype variant cap.** Pro = 2 variants per archetype per segment (already in code at `variant-engine.js:911-926`). Enterprise = 20. Don't raise Pro — would collapse tier differentiation and starve bandits.
- **Cross-archetype template pooling within store.** When templateId becomes a gene, build a shop-level template posterior pooled across archetypes, blended with cross-store meta-learning. 3-level hierarchical bandit: archetype-specific → store-level pooled → cross-store meta. Weights anneal with sample count.
- **Device personalization on Enterprise = priors, not segments.** Don't split iOS/Android/Windows into new segments — data fragmentation kills bandit performance. Add device-conditional posterior as a partial-pool prior layered on top of existing segment population.
- **Catch component:** mini-cart inline by default, cart-page banner if full cart page detected. Trigger = modal closed without claim. Lifespan = rest of session. Same theming pipeline as templates.
- **Queue tab (#7): DROPPED (2026-05-31).** Decided not to build. The queue had two sections — "Live now" (variants serving real traffic) + "On deck" (bred-but-warming + next-to-breed). Idea was Spotify-style drag-reorder. Rejected: the queue is bandit *output*, not a merchant playlist. Reordering Live-now overrides active traffic allocation and corrupts the posterior; even On-deck "next-to-breed" is recomputed each evolution cycle from fitness, so a manual order is an ephemeral nudge unless persisted as a pin. If merchant control is ever wanted, the compatible shape is **pin / exclude** (pin a variant to always-live, or veto/kill one) — NOT free reorder. For now the whole tab is cut.
- **Pro lift upsell (#5) = device-conditional upgrade nudge.** Enterprise gets device-conditional posteriors (offers personalized by device via partial-pool priors on existing segments — NOT new segments). Pro does not. The upsell analyzes the Pro store's OWN variant data, detects device cohorts behaving differently, and shows a UI card quantifying

the lift left on the table → "upgrade to Enterprise." Same "detect but don't act" pattern as the existing promo-detection upsell (`ai-decision.server.js:132`). HARD RULE: the lift figure must come from the store's real device-split data; if there isn't enough, show a qualitative nudge, never a fabricated %.

- **No merchant context survey.** Earlier idea to ask onboarding questions about existing apps was rejected — kills the few-clicks install promise. Free Shipping Bar template is the only one with real economic conflict, and it was cut.

Sprint status

Sprint 0 — DONE (commit `c6ff3a8`)

- #6 Starter gate at `apps.exit-intent.api.ai-decision.jsx` — returns 403 `upgradeRequired` if `shopRecord.plan === 'starter'`. Storefront falls through to no-modal (existing graceful handling).
- #2 Progress bar theming — `cart-monitor.js` now sniffs theme CSS custom props for cart banner / qualification banner / mini-cart CTA.

Sprint 1 — DONE (commits `092384f`, `24891da`)

- #1 Tier 1 templates: Classic Card, Top Banner, Bottom Sheet, Coupon Ticket in `extensions/exit-intent-modal/assets/modal-templates.js`. Each is a self-contained renderer with same input contract. Registry exposed as `window.ResparqTemplates.{render, list, getThemeTokens, setThemeTokens}`.
- Dispatcher branch in `exit-intent-modal.js:createModal()` — uses template registry when `mode === 'manual'` && `templateId` && `ResparqTemplates` is loaded. AI mode keeps legacy `createModal` body (will be wired in Sprint 3).
- #4 Manual picker UI in `QuickSetupTab.jsx` with SVG thumbnails. Saves to `settings.manualTemplateId` in metafield. Hidden when AI mode active.
- #6 Live preview — JSX mirrors of each template in `SettingsPreview.jsx` (`ClassicCardPreview` / `TopBannerPreview` / `BottomSheetPreview` / `CouponTicketPreview`). Dispatch on `selectedLayout` prop. Re-renders on every state change.

Bug fix on Sprint 1 (commit `24891da`)

- Storefront templates were transparent on some themes — theme-sniffing returned invalid colors. Fixed by adding `tokensFor(overrides)` merger that prefers merchant brand settings (`settings.brand*`) over auto-sniffed values. Background forced opaque via `isSafeOpaqueColor` guard, falls back to `#ffffff`.
- Discount amount didn't show in preview. Added `DiscountBadge` ("X% OFF" / "\$X OFF") above headline on Classic Card and Bottom Sheet, inline-prefix on Top Banner. Coupon Ticket already showed amount as hero. Liquid now exposes `discountPercentage` and `discountAmount` to JS.

Sprint 2 — DONE (commits `ed3e205`, `8596b0d`, `6f4de29`, `969a4db`, `655fced`, `2a49ff4`)

- #1 Tier 2 templates: Split Hero, Timer-Front, Testimonial, Scratch Reveal in `modal-templates.js` (tier:2 in registry + `MODAL_LAYOUTS`). `getAvailableLayouts()` now filters `tier <= 2` — Tier 2 is live in picker.
- JSX previews added in `SettingsPreview.jsx` (`SplitHero` / `TimerFront` / `Testimonial` / `ScratchReveal`) + thumbnails in `QuickSetupTab.jsx`.
- #3 Catch component: already shipped pre-Sprint-2 in commit `27e29b1` (mini-cart inline + cart-page banner in `cart-monitor.js`, offer pill in `exit-intent-modal.js`). Sprint 2 only added catch framing copy ("Still want your ?") — no duplicate surface built.
- Notes on the Tier 2 renderers:
 - Testimonial = merchant copy + decorative 5 stars only (no fabricated names/stats — user has zero customers, never invent social proof).
 - Scratch Reveal CTA stays clickable regardless of canvas scratch state.
 - Timer-Front is deadline-driven (not a hardcoded countdown). See fixes.

Sprint 2 follow-up fixes (post-build)

- **Timer-Front countdown** (`6f4de29`): was hardcoded ~15min. Now deadline-driven via `props.timerEndsAt` → `modal.dataset.resparqTimerEndsAt`, hours-aware (shows hh:mm:ss when $\geq 1h$). Dispatcher seeds 24h at render;

`showModal()` calls `syncTemplateTimerDeadline()` after `generateUniqueCode()` to reconcile to the real `offerExpiresAt`. Self-clears interval when modal leaves DOM.

- **Mini-cart catch line squished** (969a4db): was inserted as a flex sibling of the checkout button. Now a full-width block mounted above the cart footer via `insertMiniCartRow()` in `cart-monitor.js`.
- **Duplicate unique codes stacking** (655fced, 2a49ff4): unique-code mode minted a fresh combinable code each session; the days-long cart cookie kept old ones applied → 2-3 EXITINTE codes stacking = double/triple discount. Fix: `generateUniqueCode()` now reuses any already-applied exit code instead of minting. Detection in `findAppliedIssuedCode()` = exact match against `localStorage.exitIntentIssuedCodes` (tracked via `recordIssuedCode`, capped
 - 20. PLUS prefix match (any applied code sharing a tracked code's `PREFIX-` segment, e.g. EXITINTE10-..., catches codes minted before tracking existed). **Limitation:** a theme app extension CANNOT remove already-applied discount codes (Shopify owns that at checkout). Codes stacked before this fix need a one-time manual clear (X in checkout / empty cart). Going forward only one exit code ever lives on a cart. Root combinesWith stays `orderDiscounts:true` (`discount-codes.js`) so exit offers still stack with the store's own promos — left intact intentionally; the dedup is purely storefront-side.

Sprint 3 — IN PROGRESS

Step 1 foundation — DONE (commit c6a69fb):

- `templateId` is now a gene on `Variant` (`@default("classic-card")`), migration `20260531120000_add_template_id_to_variant` applied locally + committed.
- Gene pool: shared `TEMPLATE_IDS` (all 8 layouts) added to every archetype in `gene-pools.js` (cross-archetype pooling, per locked decision).
- Evolution engine wired in `variant-engine.js`: random create, diverse seed (spread evenly), crossover, mutation (`geneToPoolKey.templateId`), and the proven-gene seeding override all handle `templateId`.
- Meta-learning: `aggregate-gene-performance.js` aggregates `templateId` as a `geneType`, so cross-store `MetaLearningGene` populates it (covers original step 2). `determineBaseline` handles it via the generic string branch.
- Decision payload: `apps.exit-intent.api.ai-decision.jsx` now returns `decision.templateId` on both variant-bearing return shapes (discount + no-discount). Enterprise/variant path only — Pro `determineOffer` is rule-based.

Storefront render refactor — foundation DONE (commit ad01570; seams + preview harness):

- `exit-intent-modal.js` now has a single render pipeline. Extracted `buildTemplateProps(content)` (one source of truth for render props + themeOverrides; copy injected as props, never patched into DOM) and `renderTemplate(templateId, props)` (mounts, stamps legacy IDs, wires handlers, replaces any prior modal so AI can lazy re-render). Manual path `renderFromTemplateRegistry()` refactored to route through both — no behavior change.
- **Preview harness:** `?resparqPreview=<templateId>` renders any layout with representative AI-style copy and shows it immediately, bypassing the bandit, the dev-poisoned intervention threshold, and all triggers. THIS is how you verify all 8 layouts in a real theme without an exit/convert. Available ids logged to console. Use it to QA before enabling the live AI path.

Still TODO in Sprint 3:

- **Live AI render (next).** Wire AI mode to lazy-render the evolved `decision.templateId` through `renderTemplate` once the decision lands (in `getAIDecision`), behind a feature flag (default off) so it dark-launches. This replaces `updateModalWithAI`'s DOM patching. The risky copy-resolution logic in `updateModalWithAI` (placeholder replacement, showSubhead suppression, threshold copy enforcement, redirect derivation, secondary CTA, sessionStorage threshold) must be hoisted into a `resolveModalContent(decision, cartValue)` → fed to `buildTemplateProps`, with side-effects split out. Verify via the preview harness in a real theme before flipping the flag on.
- **3-level hierarchical template posterior** (archetype → store-pooled → cross-store meta, anneal with sample count). Not yet built — current wiring treats `templateId` as a flat gene like the others.
- **#5 Pro lift upsell** (device-conditional posterior + UI card). See architecture decision above for exact behavior + the no-fabricated-% rule.
- **Modal Design column on Component Analysis tab** — surfaces the `templateId` gene as a 4th leaderboard column in `app/routes/app.variants._index.jsx`. Aggregate by `v.templateId` (mirror `byHeadline` / `bySubhead` / `byCTA` at

~line 733), emit `performance.templates`, widen grid `'1fr 1fr 1fr'` → 4 cols at ~line 1448. Card should show the layout name/thumbnail (from `MODAL_LAYOUTS`) not copy text — needs a `ComponentCard type="template"` branch. Gated on step 1 (no `templateId` data until variants accumulate impressions).

- ~~#7 Queue tab~~ — DROPPED 2026-05-31 (see architecture decision).

Sprint 3 suggested order for the next instance

1. **Run the preview harness** (`?resparqPreview=<id>`) across all 8 layouts in a real theme. Confirm copy/CTA/secondary/code/timer/close render correctly. This de-risks everything downstream.
2. **Live AI render** behind a default-off flag (see TODO above). Hoist `updateModalWithAI` copy logic into `resolveModalContent`. Verify via harness, then flip flag.
3. **Component Analysis column** — low-risk admin UI, good to land anytime.
4. **3-level hierarchical posterior** — the hard algorithmic piece. Build the shop-level template posterior pooled across archetypes, blended with cross-store meta (`MetaLearningGene` already carries `templateId`). Weights anneal with sample count.
5. **#5 Pro lift upsell** — device-conditional posterior + UI card.

How the template system actually works

Storefront flow

```
Liquid snippet
  loads window.exitIntentSettings (includes templateId, brand colors, discount)
  loads modal-templates.js (registry, defines window.ResparqTemplates)
  loads exit-intent-modal.js
  loads cart-monitor.js

exit-intent-modal.js .init()
→ createModal()
  if (mode === 'manual' && templateId && ResparqTemplates)
    → renderFromTemplateRegistry()
      builds props from settings (headline, subhead, cta, amountText)
      builds themeOverrides from settings.brand* (merchant-controlled)
      calls ResparqTemplates.render(templateId, props)
      stamps IDs (#exit-intent-modal, #modal-primary-cta, #modal-secondary-cta)
        so legacy show/hide/handle code keeps working
      wires close/primary/secondary handlers
  else
    → legacy createModal body (the original ~280 lines, untouched)
```

Theme token resolution priority (modal-templates.js `tokensFor`)

<code>override.primary</code>	<code> sniffed --color-button / --color-accent-1 / --color-foreground</code>
<code>override.primaryText</code>	<code> sniffed --color-button-text / --color-background</code>
<code>override.background</code>	<code>(must pass isSafeOpaqueColor) sniffed (same) '#ffffff'</code>
<code>override.foreground</code>	<code> sniffed --color-foreground / --color-base-text '#1a1a1a'</code>
<code>override.fontFamily</code>	<code> sniffed primary-button font system stack</code>
<code>borderRadius</code>	<code>always from sniffed primary button</code>

Merchant brand settings ALWAYS win when set (≠ defaults). Auto-sniffing is fallback for first-run / unconfigured stores.

Registry contract

Each template in `modal-templates.js` is `render(props) → { overlay, modal, primaryCta, secondaryCta, closeBtn }`. Props always include:

```
{
  headline, subhead, cta, secondaryCta,
  code, amount, amountText,           // amountText is the human label "15%" / "$10"
  showSecondary, showPoweredBy,
  themeOverrides                       // optional merchant brand color override
}
```

To add a Tier 2 template:

1. Add a `renderXxx(props)` function in `modal-templates.js` following the same shape (use `tokensFor(props.themeOverrides)`, `makeCloseButton`, `makePrimaryButton`, `makeDiscountBadge`, `makePoweredBy` helpers)
2. Register in the `TEMPLATES` object at the bottom of that file
3. Add entry to `MODAL_LAYOUTS` in `app/utils/templates.js` with `tier: 2`
4. `getAvailableLayouts()` filters by `tier <= 2` today — bump the cap when ready to expose a higher tier in the picker
5. Add a matching JSX preview component in `SettingsPreview.jsx` + register in the switch inside `ModalCard` component
6. Add a thumbnail case in `LayoutThumbnail` (`QuickSetupTab.jsx`)

Known issues / gotchas

- **No-intervention loop in dev.** Adaptive intervention threshold (`app/utils/intervention-threshold.server.js`) poisoned by dev testing data. After 10 outcomes per score bucket, Thompson Sampling kicks in; if `show` arm has accumulated 0-conversion impressions, AI permanently says "no intervention." Symptom: console spam "AI decided not to show a modal." Not a bug — system working as designed. User is moving to a different test instance. **Future dev guard:** detect myshopify.com dev shops or stamped `signals.isPreview` and skip writes to `InterventionThreshold` / `InterventionOutcome` / `VariantImpression`.
- **AI mode + manual templateId.** AI mode currently ignores `templateId`. AI uses legacy `createModal` body and AI-decided copy. This is by design for Sprint 1 — Sprint 3 wires `templateId` into the bandit gene set.
- **Legacy createModal IDs.** Downstream code in `updateModalWithAI` and `closeModal` querySelects `#exit-intent-modal`, `#modal-primary-cta`, `#modal-secondary-cta`. New templates MUST receive these IDs from the dispatcher (it stamps them). Don't rename.
- **Existing MODAL_TEMPLATES constant is copy presets, NOT layouts.** Naming collision risk. The new visual templates are `MODAL_LAYOUTS` in the same file (`app/utils/templates.js`). Keep distinct.
- **17 factors claim.** Spec/marketing says "17 factors" — code has ~15-16 signals in `ai-decision.server.js:185–286`. User hasn't decided whether to bump to 17 or change the marketing. Pending.

Open product questions

1. **Scratch Reveal — RESOLVED.** Shipped in Sprint 2 (commit `ed3e205`). Canvas-based with CTA kept clickable regardless of scratch state.
2. **Catch component copy.** Should "Still want your discount?" copy be fixed or evolve via bandit? Default: fixed for now, defer to AI later.
3. **Theme app extension block** for true-native progress bar render. Not started. Deferred — current theming-via-sniff is good enough for now.

Key files

app/routes/apps.exit-intent.api.ai-decision.jsx	AI decision endpoint, plan gate
app/utils/ai-decision.server.js	Pro determineOffer + Enterprise AI
app/utils/variant-engine.js	Thompson sampling, evolution
app/utils/intervention-threshold.server.js	Adaptive show/skip threshold
app/utils/templates.js	MODAL_TEMPLATES (copy) + MODAL_LAYOUTS (visual)
app/components/settings/SettingsPreview.jsx	Live preview pane + layout previews
app/components/settings/tabs/QuickSetupTab.jsx	Manual picker UI + LayoutThumbnail
app/routes/app.settings.jsx	Settings page, state, action handler
extensions/exit-intent-modal/assets/modal-templates.js	Template registry + 4 renderers + helpers
extensions/exit-intent-modal/assets/exit-intent-modal.js	Storefront modal lifecycle (dispatcher + legacy)
extensions/exit-intent-modal/assets/cart-monitor.js	Progress bar + cart banners (theme-aware)
extensions/exit-intent-modal/snippets/exit-intent-modal.liquid	Asset loading + settings JSON

Recent commits

```
2a49ff4 Match exit codes by prefix when deduping to catch untracked stacked codes
655fced Dedup unique discount codes to prevent stacking on persistent cart
969a4db Fix squished mini-cart catch line in flex-footer themes
6f4de29 Fix Timer Front countdown to match real promotion window
8596b0d Sprint 2: catch framing on dismissal-recovery surfaces
ed3e205 Sprint 2: 4 Tier 2 modal templates (Split Hero, Timer Front, Testimonial, Scratch Reveal)
27e29b1 Stack exit offers with store promos; stop auto-pausing AI during promos
24891da Fix transparent storefront modal + add discount badge to preview/storefront
092384f Sprint 1: 4 modal-layout templates + manual picker + live preview
c6ff3a8 Sprint 0: gate Starter from AI mode + native theming for cart progress bar
```

When user says "ready" or "go" next time

Next concrete work is **Sprint 3** (the bandit/AI integration — biggest chunk, touches DB schema + evolution engine + admin UI). Decisions are locked above ("Architecture decisions made — don't relitigate"). Suggested order:

1. **templateId as a gene** — Prisma migration adding `templateId` to `Variant`; wire into `variant-engine.js` (Thompson sampling / breeding) as a gene. Foundation for everything else. Build the 3-level hierarchical template posterior (archetype → store-level pooled → cross-store meta), weights anneal with sample count.
2. **Populate `MetaLearningGene` with `templateId`** — so cross-store meta-learning has the new gene.
3. **Modal Design column on Component Analysis tab** — surfaces the new `templateId` gene as a fourth leaderboard column in `app/routes/app.variants._index.jsx`. Aggregate by `v.templateId` (mirror `byHeadline` / `bySubhead` / `byCTA` at ~line 733), emit `performance.templates`, widen grid `'1fr 1fr 1fr'` → 4 cols at ~line 1448. Card shows layout name/thumbnail (from `MODAL_LAYOUTS`), not copy text — needs a `ComponentCard type="template"` branch. Gated on step 1 (no `templateId` on `Variant` until the migration lands).
4. **#5 Pro lift upsell** — device-conditional posterior as a partial-pool prior layered on existing segment population (NOT new segments — data fragmentation kills bandits). Plus the UI upsell card.

(Queue tab #7 dropped 2026-05-31 — see Architecture decisions.)

Don't restart the architectural conversation — decisions above are locked.

Open carry-over items (still unresolved going into Sprint 3)

- **Dev-data poisoning guard** (see Known issues): InterventionThreshold can get poisoned by dev testing → "AI decided not to show." Future guard: detect myshopify.com dev shops or `signals.isPreview` and skip writes to `InterventionThreshold / InterventionOutcome / VariantImpression`.
- **17 factors claim**: code has ~15-16 signals; marketing says 17. Unresolved.
- **Scratch Reveal cost / catch copy bandit / native theme block** — see "Open product questions" below. Scratch Reveal shipped in Sprint 2.